

Arrays in Python

What is an Array?

	A	B	C
1	Name	Income (€)	Experience (yrs)
2	Bill	50000,00	12,00
3	Ann	51000,00	14,00
4	Dave	38000,00	10,00
5	Jill	47000,00	11,00

- **Array:** A set of numerical variables represented in a tabular form.
- **Rows:** Represent entities (e.g., persons).
- **Columns:** Represent measured properties.

Why Use numpy arrays and not lists?

- **Efficiency:** Numpy arrays are more efficient than Python lists for numerical data, because in numpy storage is contiguous
- **Speed:** Numpy arrays can be 100x or more faster than lists.
- **Homogeneity:** Numpy arrays can only hold numerical data.
 - limitation, but beneficial for performance

Creating Arrays

1D Arrays (Vectors)

```
import numpy as np

#A row vector
a = np.array([1,2,3])
print("a = ", a)
#A column vector
b = np.array([[1],[2],[3]])
print("b = \n",b)
```

```
a =  [1  2  3]
b =  [[1]
      [2]
      [3]]
```

2D Arrays (Matrices)

```
A = np.array([[1, 2, 34], [3, 4, 45]])  
print(a)
```

```
[[ 1  2 34]  
 [ 3  4 45]]
```

Special Arrays

- Zeros Matrix

```
Z = np.zeros((2,3))  
print("Matrix of zeros: ")  
print(Z)
```

```
Matrix of zeros:  
[[0. 0. 0.]  
 [0. 0. 0.]
```

- Random Matrix

```
R = np.random.randint(low=2, high=6, size=(5, 3))  
print("Random matrix:")  
print(R)
```

Random matrix:

```
[[3 2 3]  
 [3 4 2]  
 [3 5 5]  
 [5 3 3]  
 [3 2 4]]
```

Using a formula to fill values

- Sometimes array element value depends on its indices `i` and `j`.
 - use for loops or `numpy.fromfunction`
 - `numpy.fromfunction` should be faster

for loops

```
from numpy import zeros
def lehmer(n):
    A = zeros((n,n))
    for i in range(n):
        for j in range(n):
            A[i,j] = min(i + 1, j + 1)/max(i + 1, j + 1)
    return A

print(lehmer(4))
```

```
[[1.          0.5         0.33333333 0.25         ]
 [0.5         1.          0.66666667 0.5         ]
 [0.33333333 0.66666667 1.          0.75         ]
 [0.25         0.5         0.75         1.          ]]
```

numpy.fromfunction

```
A = np.fromfunction(lambda i, j: np.minimum(i + 1, j + 1)/np.maximum(i + 1, j + 1), (4, 4))  
print(A)
```

```
[[1.          0.5         0.33333333 0.25         ]  
 [0.5         1.          0.66666667 0.5         ]  
 [0.33333333 0.66666667 1.          0.75         ]  
 [0.25         0.5         0.75         1.          ]]
```

Array Indexing and slicing

1D Array Indexing and slicing

```
x = np.array([13, 42, 23, 14, 74, 15])  
print(x[0])  # First element  
print(x[0:2])  # First two elements
```

```
13  
[13 42]
```

2D Array Indexing

```
import numpy as np
A = np.array([[2, 3, 4], [5, 6, 7]])
print(A[0]) # The first row
print(A[0,:]) #Also the first row
print(A[0:2, 1:3]) # Submatrix, rows 0,1 and cols 1,2
```

```
[2 3 4]
[2 3 4]
[[3 4]
 [6 7]]
```

Array Operations

Basic Statistics

```
x = np.array([1,43,2,4,54,5,3,55])  
print(x.max()) # Largest element  
print(x.min()) # Smallest element  
print(x.mean()) # Average  
print(x.std()) # Standard deviation
```

```
55  
1  
20.875  
23.342223008959536
```

Matrix Operations

Matrix product is calculated in the following way:

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} + a_{12}b_{21} & a_{11}b_{12} + a_{12}b_{22} \\ a_{21}b_{11} + a_{22}b_{21} & a_{21}b_{12} + a_{22}b_{22} \end{bmatrix}$$

```
A = np.array([[2, 3], [4, 5]])  
B = np.array([[1, 2], [3, 4]])  
C = A @ B # Matrix product  
print(C)
```

```
[[11 16]  
 [19 28]]
```

- **Transpose:**
 - swaps the columns and rows

```
A = np.matrix([[1,2],[3,4],[5,6]])
A_transpose = A.transpose()
print("A =\n",A)
print("")
print("Transpose of A =\n",A_transpose)
```

```
A =
[[1 2]
 [3 4]
 [5 6]]

Transpose of A =
[[1 3 5]
 [2 4 6]]
```

Solving Linear Systems

$$\begin{cases} x_1 + 5x_2 + x_3 = 24 \\ 7x_1 + \frac{1}{5}x_2 + 4x_3 = 0 \\ x_1 + 3x_2 + 2x_3 = 12 \end{cases}$$

```
A = np.array([[1, 5, 1], [7, 1/5, 4], [1, 3, 2]])  
b = np.array([24, 0, 12])  
xyz = np.linalg.solve(A, b)  
print(xyz)
```

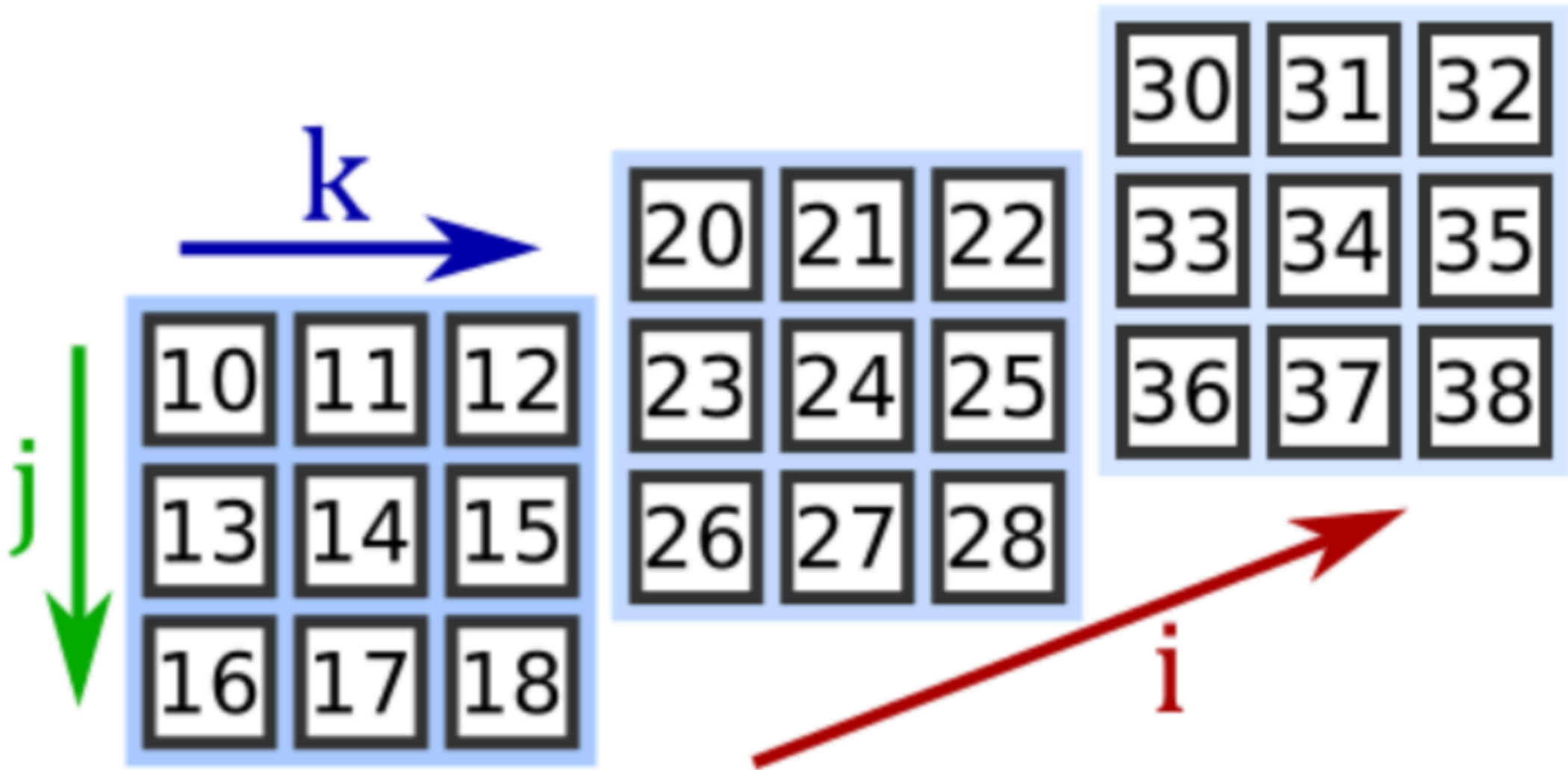

Broadcasting

- **Broadcasting:** Allows arithmetic operations between arrays of different shapes.
- **Example:**

```
x = np.array([1, 2, 3])  
A = np.array([[5, 6, 7], [8, 9, 10]])  
B = A + x  # x is broadcasted to [[1, 2, 3], [1, 2, 3]]
```

n dimensional arrays

- The number of dimensions in arrays is not limited.
- The number of indices needed to select an element is called the **dimension** of the array.
- Three dimensional array can be imagined to be a stack of two dimensional arrays.



```
A = np.array([[[10, 11, 12], [13, 14, 15], [16, 17, 18]],  
              [[20, 21, 22], [23, 24, 25], [26, 27, 28]],  
              [[30, 31, 32], [33, 34, 35], [36, 37, 38]]])
```

```
print(A.shape)
```

```
(3, 3, 3)
```

- Three indices are used to select elements in an array.

```
A = np.array([[[10, 11, 12], [13, 14, 15], [16, 17, 18]],  
              [[20, 21, 22], [23, 24, 25], [26, 27, 28]],  
              [[30, 31, 32], [33, 34, 35], [36, 37, 38]]])  
print("A[0,0,0] = ",A[0,0,0])  
print("A[1,2,0] = ",A[1,2,0])  
print("A[1,:,:] =\n", A[1,:,:]) #The matrix in the middle  
print("A[1,2] = ",A[1,2]) #The same as A[1,2,:]  
print("A[1,:,2] =",A[1,:,2])
```

```
A[0,0,0] = 10  
A[1,:,:] =  
  [[20 21 22]  
   [23 24 25]  
   [26 27 28]]  
A[1,2,0] = 26  
A[1,2] = [26 27 28]  
A[1,:,2] = [22 25 28]
```

Slicing in higher dimensions

```
A = np.array([[[10, 11, 12], [13, 14, 15], [16, 17, 18]],  
              [[20, 21, 22], [23, 24, 25], [26, 27, 28]],  
              [[30, 31, 32], [33, 34, 35], [36, 37, 38]]])
```

```
print(A[:2,1:,:2])
```

```
#The first two planes with :2,
```

```
#after that from the second row to the bottom with 1:
```

```
#and the first two rows with :2
```

```
print("A=\n",A)
```

```
[[[13 14]  
  [16 17]]
```

```
[[23 24]  
 [26 27]]]
```